

LC 32: Longest Valid Parentheses

© Authored by: Arpit Raj, LNMIIT Jaipur | Date: September 11, 2026

Asked in: DE Shaw, MMT, SO TI, Morgan Stanley

Examples

1. $s = "()" \rightarrow \text{ans} = 2$
2. $s = "()()()" \rightarrow \text{ans} = 6$
3. $s = ")()()()" \rightarrow \text{ans} = 4$
4. $s = "()(())" \rightarrow \text{ans} = 2$
5. $s = "()(())" \rightarrow \text{ans} = 6$ (because $((()))$ itself is valid)

DP Approach

$dp[i]$ → will give max length of valid substring ending at index i

Case 1: () this type

then ... [] ()

 ↑ ↑ ↑
 (i-2) i-1 i

hence: $2 + dp[i-2]$

Case 2:)) this type, see Eg 5

then, need to find matching (for last)

so)) → start index of inner valid substring is $i - dp[i-1]$

↑ ↑
(i-1) i

and we need to find matching (

so $j = i - dp[i-1] - 1$

Now If $s[j] == '('$ then

$dp[i] = dp[i-1] + 2$ → just matched ' (' and ') '

additionally there might already be valid pairs before this pair

hence $dp[i] += dp[j-1]$

at every step need to do $ans = \max(ans, dp[i])$

C++ Implementation

```
class Solution {
public:
    int longestValidParentheses(string s) {

        int n = s.size();
        vector<int> dp(n, 0);

        int ans = 0;

        for(int i = 1; i < n; i++) {
            if(s[i] == ')') {
                if(s[i-1] == '(') {

                    dp[i] = 2;
                    if(i - 2 >= 0)
                        dp[i] += dp[i-2];
                }
            }
            else {
            }
```

```
        int j = i - dp[i-1] - 1;

        if(j >= 0 && s[j] == '(') {
            dp[i] = dp[i-1] + 2;

            if(j - 1 >= 0)
                dp[i] += dp[j-1];
        }
    }
    ans = max(ans, dp[i]);
}

return ans;
}

};
```